

Nombre (s) : Darío Alberto Sánchez Perzabal Santiago Patricio Gómez Ochoa Ximena Ortiz Gómez María José Croche Álvarez		Matrícula (s): A01738266 A01735171 A01735100 A01734561
Nombre del curso: Fundamentación de Robótica	Nombre del profesor (es): Juan Manuel Ahuactzin Larios Rigoberto Cerino Jiménez Alfredo García Suárez	
Módulo:	Actividad: Actividad 2.4 Posición y velocidad de un Motor DC	
Fecha: 11 de marzo del 2026		

Resumen

Este reporte habla sobre el desarrollo de un sistema que consiste de un motor DC, que además tiene un encoder, un Puente H, una ESP32 en la que se utilizó Arduino que como principales objetivo se tiene calcular la posición y velocidad del motor. Para ello, fue necesario primero comprender cómo mide los pulsos un encoder de un motorreductor. En este caso se usó un encoder de cuadratura, que trabaja con dos canales, A y B. Igualmente es necesario obtener los PPR, que serán la resolución del motor, para ello se hizo uso de Arduino en la ESP32. Una vez con esta información se calcula la posición y eventualmente la velocidad del mismo motor. Igualmente se implementaron interrupciones, las cuales entran en acción una vez se detecte un cambio en el sistema.

Objetivos

En esta práctica, el objetivo base fue obtener los pulsos del motor, pues una vez sabiendo esta información, se puede saber la resolución del motor, información relevante para diferentes mediciones que se quieran obtener, un ejemplo de ello son precisamente la posición y la velocidad, objetivos principales de la práctica. Igualmente, la comprensión e interpretación de las señales del encoder son base de la práctica, pues se puede observar, con ayuda de un osciloscopio, como se comportan los sentidos del giro del motor.

Introducción

En sistemas de robótica, el control preciso es extremadamente importante para garantizar el buen funcionamiento de actuadores como lo son los motores. Por ende para lograr este control es importante contar con dispositivos de retroalimentación que nos permitan medir variables como la posición, la velocidad angular y el sentido de giro de un motor. Por ende en los motores de corriente directa (DC) el sensor más usado son los encoders de cuadratura pues nos permiten realizar mediciones precisas de la posición del eje del motor y por ende conocer su velocidad.

Hablando específicamente de los encoders de cuadratura, estos son los sensores más utilizados para medir la posición, velocidad y aceleración de los motores. Por ende, el encoder de cuadratura es un tipo de encoder rotativo incremental que tiene la capacidad de indicar la posición, la velocidad y la dirección del eje de un motor. Este encoder genera dos señales de pulsos que están desfasadas una de la otra 90° , señales a las cuales se les conoce como señal A y señal B. Estos dos sensores se van activando y desactivando en una secuencia que permite saber la dirección y el desplazamiento del motor a través del encoder. En la figura 1 podemos observar ambos canales del encoder de cuadratura los cuales también nos permiten determinar el sentido de giro a través del desfase de las señales [1].

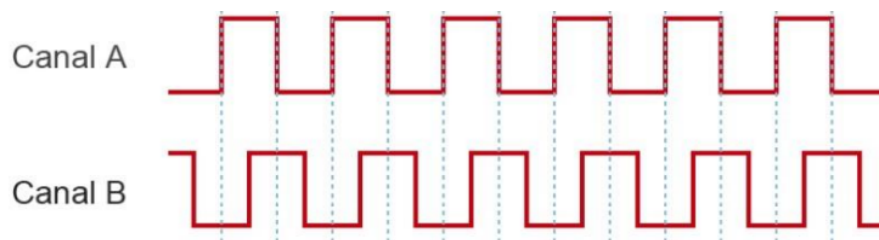


Figura 1. Señales A y B de un encoder de cuadratura

Tomando en cuenta lo anteriormente mencionado, a través de los pulsos generados por la señal A y B de un encoder podemos conocer la resolución del encoder, la cual es una medida en pulsos por revolución (PPR) que genera el encoder cada 360° y con la cual podemos determinar la posición angular del motor. De igual manera mediante el conteo de pulsos generados por el encoder se puede estimar la posición del eje tanto en revoluciones como en grados, lo que nos permite realizar un seguimiento correcto y preciso del motor [1].

Consecuentemente, como se mencionó anteriormente a través del encoder se puede realizar el cálculo de la posición del motor se obtiene contando los pulsos del encoder generados por la señal A y la señal B [2]. Por ende considerando que el encoder genera un número de pulsos por revolución, podemos determinar la cantidad de revoluciones realizadas

por el eje del motor usando la siguiente ecuación a través de los números de pulsos y los pulsos por revolución:

$$revoluciones = \frac{\text{Número de pulsos}}{PPR} \quad (1)$$

A partir del valor de las revoluciones podemos calcular la posición angular en grados mediante la siguiente expresión [2]:

$$\theta = \frac{\text{Número de pulsos}}{PPR} * 360 \quad (2)$$

De igual manera para calcular la velocidad angular del motor a partir del cambio en la posición en un intervalo de tiempo. El método Euler se puede hacer mediante el método Euler, el cual aproxima la derivada de la posición angular respecto al tiempo a través de la siguiente expresión donde θ_k corresponde a la posición angular actual, θ_{k-1} representa la posición angular en el instante anterior y Δt es el intervalo de tiempo entre ambas mediciones [2]:

$$\omega = \frac{\theta_k - \theta_{k-1}}{\Delta t} \quad (3)$$

Es por ello que tomando como base la ecuación anterior, a partir de esta ecuación podemos obtener la velocidad angular utilizando un método numérico y utilizando las mediciones de la posición proporcionadas por el encoder.

Finalmente es importante definir qué son las interrupciones y cómo se usan en una ESP32 ya que son conceptos usados en esta actividad. Las interrupciones permiten que el microcontrolador detecte y responda a cambios específicos en los pines de entrada incluso cuando se está ejecutando el programa principal. Cuando ocurre una interrupción el programa principal se detiene momentáneamente y ejecuta la función callback. Una vez completada la función callback, el programa principal continúa su ejecución [3]. En esta práctica las interrupciones se configuraron para activarse cuando ocurre un cambio en el canal del encoder, por lo que cada vez que se detecta dicho evento, se ejecuta la rutina de servicio de interrupción la cual actualiza el contador de pulsos y se evalúa el canal B para determinar el sentido de giro del motor.

Solución al problema

Metodología implementada

Esta actividad consistía en la programación de interrupciones y estimar la velocidad de un motor de corriente directa mediante un encoder de cuadratura, un ESP32, un puente H y arduino. Además, como se mencionó anteriormente, el objetivo de esta actividad era realizar el conteo de pulsos del encoder de cuadratura con el fin de estimar la posición y velocidad del motor. Es por ello que la metodología implementada para conseguir los objetivos fueron realizadas en 3 etapas. La primera etapa consistió en la determinación del número de pulsos del encoder, la segunda etapa consistió en la implementación de un sistema de medición de posición y velocidad y finalmente la tercera etapa consistió en la verificación experimental de la velocidad del motor.

Hablando específicamente de la primera etapa la cual fue la determinación del número de pulsos del encoder, se tomó como base un programa en arduino cuyo objetivo fue determinar el número de pulsos generados por el encoder del motor durante una revolución completa de la flecha de salida. Para ello, como se muestra en la figura 2, se utilizó un encoder de cuadratura conectado a los siguientes pines GPIO de nuestra ESP32.

Señal	Pin ESP32
Canal A del encoder	GPIO 25
Canal B del encoder	GPIO 26

Figura 2. Señales A y B de un encoder de cuadratura y pines en los que se encuentra de la ESP32

Por ende, nuestro conteo de pulsos se realizó mediante interrupciones externas, lo cual nos permitió detectar cada cambio en las señales del encoder sin perder ninguna información. La interrupción se configuró de la siguiente manera en el setup de nuestro código de arduino:

```
void setup ()
```

```
{ Serial.begin(115200);
```

```
pinMode(EncA, INPUT_PULLUP); // Señal A del encoder como entrada con pull-up
```

```
pinMode(EncB, INPUT_PULLUP); // Señal B del encoder como entrada con pull-up
```



```
attachInterrupt(digitalPinToInterrupt(EncA), Encoder, CHANGE); // Asignar la función
Encoder a la interrupción de cambio en la señal A

}
```

Como podemos observar, cada vez que se detecta un cambio en la señal A, se ejecuta la función de interrupción Encoder, la cual compara el estado de los canales A y B para poder determinar cual es el sentido de giro de nuestro motor y actualizar el contador de pulsos. Por ende la lógica implementada en el código fue la siguiente:

```
void IRAM_ATTR Encoder()
{
  BSet = digitalRead(EncB);
  ASet = digitalRead(EncA);
  if (BSet == ASet)
  {
    contador++;
  }
  else
  {
    contador--;
  }
}
```

Como podemos observar cuando ambas señales presentan el estado mismo lógico se incrementa el contador, mientras que cuando son distintas se decrementa el mismo contador. Este método permite conocer no solo el número de pulsos generados, sino también la dirección de rotación del motor. Es por ello que durante esta etapa se realizó el conteo de los pulsos generados durante una vuelta completa del eje del motor obteniendo como resultado 4191 pulsos por revolución.

Tomando en cuenta lo anterior, una vez determinada la resolución del encoder, se utilizó un segundo programa en arduino que permite calcular la posición angular, el número de revoluciones y la velocidad del motor en RPM a través de un potenciómetro. De igual forma, en este programa el motor se controla mediante un puente H el cual permite determinar el sentido de giro del motor usando los pines digitales del ESP32 y dependiendo de la combinación lógica de estos pines, el motor puede girar sentido horario o antihorario. A continuación se presentan la conexión de pines digitales de la ESP32 conectados al puente H:

Pin	Función
GPIO23	Dirección 1 (In1)
GPIO22	Dirección 2 (In2)

Figura 4. Pines de la ESP32 donde está conectado el puente H y sus funciones

Es por ello que la lógica implementada fue la siguiente y la cual nos ayudó a determinar la acción que haría el motor de acuerdo al estado de los pines. Lo anterior se muestra en la siguiente figura:

In1	In2	Acción del motor
HIGH	LOW	Giro a la derecha
LOW	HIGH	Giro a la izquierda
LOW	LOW	Motor detenido

Figura 5. Estado de los pines del puente H y la acción del motor ante estos estados

Posteriormente la velocidad del motor se controló utilizando una señal PWM generada por el LEDC de nuestro ESP32. Por ende en nuestro código el valor de PWM se obtiene de la lectura analógica del potenciómetro conectado al pin GPIO 34 y la lectura del ADC del ESP32 genera valores entre 0 y 4095, los cuales se mapean al rango del PWM de 0 a 255.

//Función para leer los valores de un potenciómetro para controlar los motores por PWM

void lectura ()

{

 // leer señal analógica de entrada:

 pot = analogRead(variador);

 // mapear el rango de la salida:

 pwm = map(pot, 0, 4095, 0, 255);

 // cambiar el valor de salida:

 ledcWrite(PWM1_Ch, pwm);

 //analogWrite(EnA, pwm);

}

Luego se realizó el cálculo de la posición del motor utilizando el número de pulsos detectados por el encoder y la resolución previamente determinada en el código anterior. La conversión fue realizada mediante la función pose en la cual el contador representa el número de pulsos acumulados. Además, resulta importante destacar que cuando el número de pulsos alcanza el valor correspondiente a una revolución completa que en nuestro caso son 4191 pulsos, se incrementa el contador de revoluciones:

```
void pose ()
{
  if (encoderDirection)
  {
    posicion = contador * resolucion; //Convertir a grados
    if (contador >= pulsos) //Contar por revoluciones
    {
      revoluciones++;
      contador = 0;
    }
  }
  else
  {
    posicion = contador * resolucion; //Convertir a grados
    if (contador <= -pulsos) //Contar por revoluciones
    {
      revoluciones--;
      contador = 0;
    }
  }
  //Cálculo de la velocidad mediante un delta de tiempo
  velocidad = 60000000/(pulsos * delta_tiempo); // Convertir a segundos nuevamente
multiplicando por 60000000
  //Se pregunta por la velocidad, cuando hay una inversion de giro, para hacerla positiva
  if (velocidad < 0)
    velocidad = abs(velocidad);
  encoderDirection = false; // Reiniciar la variable después de usarla
}
```

Como penúltimo paso, la velocidad del motor se estimó utilizando la diferencia de tiempo entre dos pulsos consecutivos del encoder para medir intervalos de tiempo en microsegundos. Por ende, dentro de la rutina de interrupción se calcula el intervalo de tiempo entre pulsos y posteriormente la velocidad en revoluciones por minuto en tiempo real, lo anterior se hace en el siguiente fragmento de código:

```

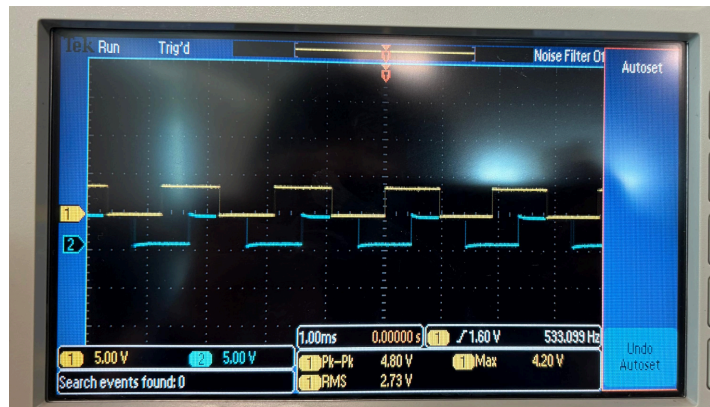
void IRAM_ATTR Encoder()
{
  BSet = digitalRead(EncB);
  ASet = digitalRead(EncA);
  if (BSet == ASet)
  {
    contador++;
    encoderDirection = true;
  }
  //Si ambas señales leídas son distintas, el motor gira en sentido horario
  //y se decrementa un contador para saber el número de lecturas
  else
  {
    contador--;
    encoderDirection = false;
  }
  tiempo_act = micros();
  delta_tiempo = tiempo_act - tiempo_ant;
  tiempo_ant = tiempo_act;
}

```

Finalmente, para validar el funcionamiento del sistema se realizó una prueba experimental donde comparamos el resultado dado por el programa con el conteo real de las revoluciones por minuto de nuestro motor. Esto permitió confirmar si el sistema de medición basado en el encoder y el cálculo de la velocidad de la ESP32 eran correctos, lo cual se explica con mayor detenimiento en la sección de resultados.

Resultados

Para esta práctica, se logró estimar la posición y velocidad en RPM de un motorreductor a través de un sensor *Encoder de cuadratura*. Como se ha mencionado anteriormente, una de las particularidades de esta clase de sensor es que al comparar las respuestas de sus dos canales, se puede diferenciar el sentido en el cual está girando el motor. El primer paso para estimar la posición y la velocidad del motorreductor fue analizar la resolución del encoder y el número de pulsos que este entrega en una vuelta. Para ello, se inició por comparar la respuesta de los canales *A* y *B* del encoder.



c1. Respuesta de los canales A (amarillo) y B (azul) del encoder de cuadratura en un osciloscopio cuando el motorreductor gira en el sentido del reloj.

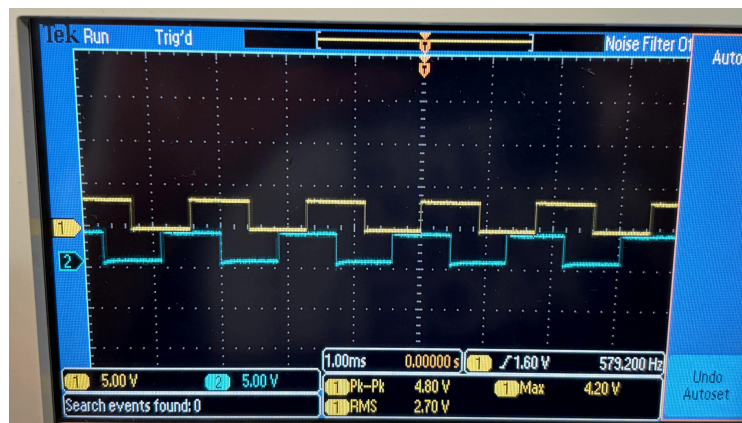
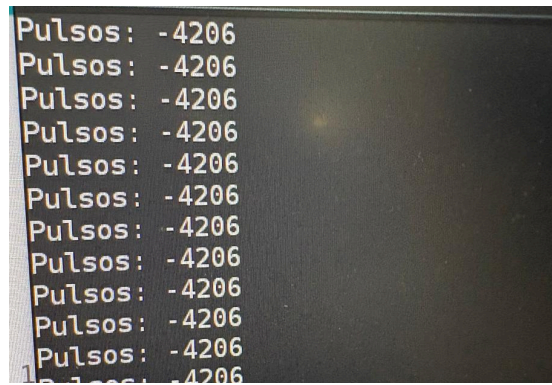


Imagen 2. Respuesta de los canales A (amarillo) y B (azul) del encoder de cuadratura en un osciloscopio cuando el motorreductor gira contrario al sentido del reloj.

Si se observa cuidadosamente la respuesta de ambos canales representados en la **Imagen 1** y la **Imagen 2**, se podrá observar que ambas respuestas tienen un desfase de 90° una de la otra que ayuda en la identificación del sentido de giro. En la **Imagen 1**, cuando el canal A cambia de estado (por ejemplo, de bajo a alto), el canal B está en el estado contrario. Por otro lado, en la **Imagen 2** ambas señales están en el mismo estado cuando el canal A cambia de estado. Esto muestra la respuesta de ambos canales al cambiar el sentido de giro.

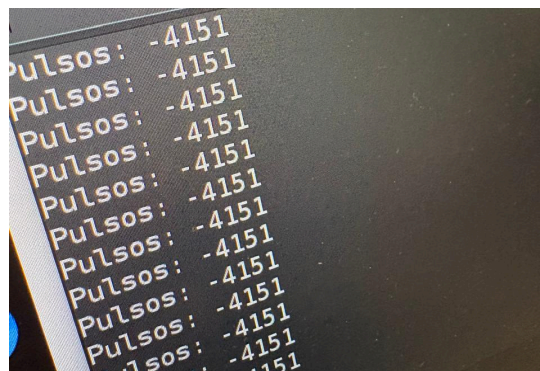
Una vez analizada esta cuestión, se prosiguió con el análisis de los pulsos por revolución que da el encoder. En este caso, se realizaron 5 mediciones de manera heurística, en las cuales se contaron los pulsos que el encoder entregaba al realizar una vuelta completa.



A close-up photograph of a serial monitor screen. The screen displays a list of pulse counts, each preceded by the label 'Pulsos:'. The values are consistently -4206. The text is white on a black background.

```
Pulsos: -4206
Pulsos: -4206
Pulsos: -4206
Pulsos: -4206
Pulsos: -4206
Pulsos: -4206
Pulsos: -4206
Pulsos: -4206
Pulsos: -4206
Pulsos: -4206
Pulsos: -4206
Pulsos: -4206
Pulsos: -4206
```

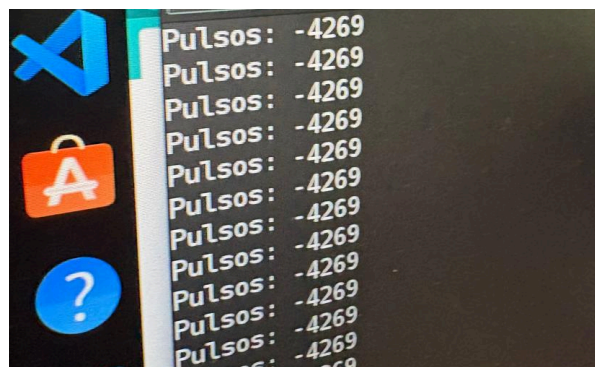
Imagen 3. Primer resultado de puerto serial de la ESP32 al ejecutar el contador de pulsos.



A close-up photograph of a serial monitor screen. The screen displays a list of pulse counts, each preceded by the label 'Pulsos:'. The values are consistently -4151. The text is white on a black background.

```
Pulsos: -4151
Pulsos: -4151
Pulsos: -4151
Pulsos: -4151
Pulsos: -4151
Pulsos: -4151
Pulsos: -4151
Pulsos: -4151
Pulsos: -4151
Pulsos: -4151
Pulsos: -4151
Pulsos: -4151
Pulsos: -4151
```

Imagen 4. Primer resultado de puerto serial de la ESP32 al ejecutar el contador de pulsos.



A close-up photograph of a serial monitor screen. The screen displays a list of pulse counts, each preceded by the label 'Pulsos:'. The values are consistently -4269. To the left of the text, there are three icons: a blue 'X' icon, an orange shopping bag icon with a white 'A', and a blue circle icon with a white question mark. The text is white on a black background.

```
Pulsos: -4269
Pulsos: -4269
Pulsos: -4269
Pulsos: -4269
Pulsos: -4269
Pulsos: -4269
Pulsos: -4269
Pulsos: -4269
Pulsos: -4269
Pulsos: -4269
Pulsos: -4269
Pulsos: -4269
Pulsos: -4269
```

Imagen 5. Primer resultado de puerto serial de la ESP32 al ejecutar el contador de pulsos.

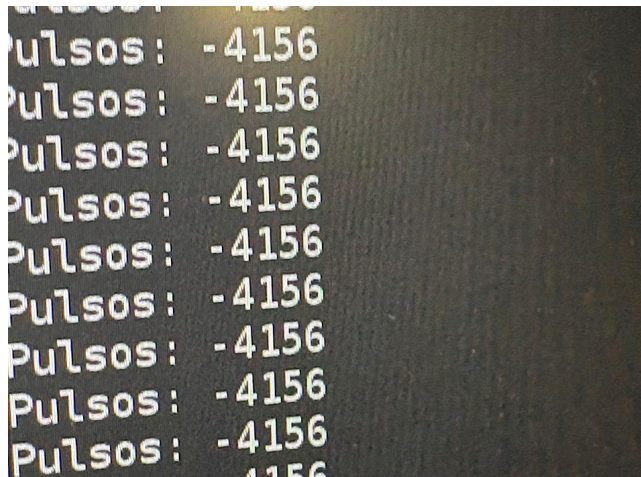


Imagen 6. Primer resultado de puerto serial de la ESP32 al ejecutar el contador de pulsos.

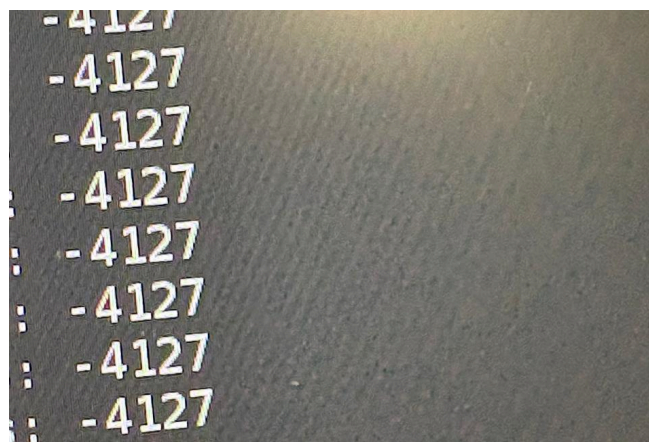


Imagen 7. Primer resultado de puerto serial de la ESP32 al ejecutar el contador de pulsos.

Como se puede ver en las **Imágenes 3 - 7**, los pulsos medidos estaban alrededor de 4200 en la mayoría de los casos. Al sacar la media de las cinco mediciones se obtuvo que el número de pulsos era de 4191, lo cual permitió sacar la resolución del encoder mediante la expresión $Resolución = \frac{360^\circ}{4191} = 0.0858983536 \text{ grados}$.

Ya que se obtuvo la resolución del encoder, se utilizó esta junto con el número de pulsos para sacar un estimado de la posición y la velocidad. Se tuvo en cuenta que al ser un motorreductor de 6V, el puente H que ayudó al control del motor se alimentó de una fuente de voltaje de precisamente 6VDC. Considerando lo anterior, se obtuvo el siguiente resultado.

/dev/ttyACM0				
Posicion:	42	Revoluciones y	172.05	Grados
A una velocidad estimada de:	22.00		RPM	
Posicion:	42	Revoluciones y	185.37	Grados
A una velocidad estimada de:	21.00		RPM	
Posicion:	42	Revoluciones y	198.68	Grados
A una velocidad estimada de:	22.00		RPM	
Posicion:	42	Revoluciones y	212.00	Grados
A una velocidad estimada de:	21.00		RPM	
Posicion:	42	Revoluciones y	225.31	Grados
A una velocidad estimada de:	22.00		RPM	

Imagen 8. Estimación de la velocidad máxima RPM y posición del motor.

/dev/ttyACM0				
Posicion:	52	Revoluciones y	41.75	Grados
A una velocidad estimada de:	0.00		RPM	
Posicion:	52	Revoluciones y	42.26	Grados
A una velocidad estimada de:	1.00		RPM	
Posicion:	52	Revoluciones y	42.61	Grados
A una velocidad estimada de:	0.00		RPM	
Posicion:	52	Revoluciones y	43.12	Grados
A una velocidad estimada de:	0.00		RPM	
Posicion:	52	Revoluciones y	43.46	Grados
A una velocidad estimada de:	0.00		RPM	

☒ Autoscroll ☐ Show timestamp

Imagen 9. Estimación de la velocidad mínima en RPM y posición del motor.

Las **Imágenes 8-9** corresponden a la estimación de la velocidad en RPM que puede alcanzar el motorreductor, al igual que los grados que este ha rotado en su movimiento. Cuando el conteo de estos últimos alcanza un valor de 360° o más, este contador se reinicia y se añade 1 a la cantidad de revoluciones que el motor ha dado. La **Imagen 8** muestra la velocidad y posición del motor a su máxima capacidad, indicando que alcanza una velocidad de aproximadamente 22 RPM. Nótese que el cambio en la medición de los grados que ha rotado es bastante alto, mostrando que el motor gira a su máxima velocidad. Por su parte, la **Imagen 9** muestra al motor operando a la mínima capacidad detectable por el encoder. Aunque la velocidad estimada es de 0.0 o 1.0 RPM, aun hay una mínima razón de cambio en el conteo de grados, indicando que aún hay movimiento (casi imperceptible).

Lo siguiente es un video demostrativo de que el motor alcanza físicamente la velocidad máxima estimada:

[Video 1] https://www.youtube.com/watch?v=_Qw7fPOL1Uk

En este se observa que visualmente alcanza 23 revoluciones en un minuto, lo cual se acerca bastante al valor previamente estimado.

Conclusiones

El desarrollo de esta práctica demostró la eficacia de utilizar un encoder de cuadratura junto con un microcontrolador ESP32 para calcular con precisión la posición, la velocidad y el sentido de giro de un motor DC. Los hallazgos más destacados obtenidos tras la experimentación son los siguientes:

- La determinación empírica de 4191 pulsos por revolución (PPR) dotó al sistema de una excelente resolución de aproximadamente 0.086 grados por pulso, permitiendo un cálculo de posición angular sumamente detallado.
- La configuración de interrupciones de hardware fue una implementación fundamental para evitar la pérdida de datos durante los cambios rápidos de estado en los canales A y B del encoder.
- El análisis del desfase de 90° entre las señales emitidas por el encoder comprobó ser un método lógico y funcional para identificar de manera constante el sentido de giro del motorreductor.
- El modelo matemático utilizado para estimar la velocidad mediante el diferencial de tiempo en microsegundos demostró un alto grado de exactitud, ya que el tope de 22 RPM calculado por el código tuvo una variación mínima frente a las 23 RPM reales comprobadas en video y que además son variantes dependiendo del voltaje de entrada del motor.
- La integración conjunta del puente H y las señales PWM facilitó la manipulación controlada del actuador, permitiendo registrar variaciones de velocidad desde un estado casi imperceptible hasta su máxima capacidad a 6V.

Referencias

1. Sásig, E. (s.f.). Robótica con Arduino.
https://roboticoss.com/wp-content/uploads/2021/08/Guia_de_encoders_para_robotica.pdf Principios de funcionamiento del encoder cuadratura de efecto Hall para medición de velocidad en motores de corriente directa.
2. Punto Flotante (s.f.). Principios de funcionamiento del encoder cuadratura de efecto Hall para medición de velocidad en motores de corriente directa.
<https://www.puntoflotante.net/FUNCIONAMIENTO-ENCODER-CUADRATURA-EFECTO-HALL.htm>
3. Llamas, L. (s.f.). Cómo usar interrupciones en un ESP32.
<https://www.luisllamas.es/esp32-interrupciones-hardware/>